



Norwegian University of
Science and Technology

Practical Anonymous Communication with Homomorphic Encryption

Angelo, Imre

Submission date: June 2026

Main supervisor: Associate Professor Park, Jeongeun, NTNU

Norwegian University of Science and Technology
Department of Information Security and Communication Technology

Problem description

Many anonymous communication systems have been proposed and implemented with the goal of allowing users to exchange messages over a network without revealing who is communicating with whom. Most of these designs rely on the *threshold model* to provide anonymity, wherein some components of their infrastructure must be behaving honestly. Systems with stronger anonymity guarantees generally suffer from poor scalability or are impractical to instantiate.

This thesis examines a new construction called a (Somewhat) Practical Anonymous Router (sPAR), which addresses the trust and practicality concerns of existing systems. The construction relies only on well-established Fully Homomorphic Encryption (FHE) schemes, making it realistically implementable. While sPAR presents a promising new avenue for constructing anonymous communication systems, it remains a theoretical construction. A concrete implementation and a deeper investigation is therefore needed to evaluate its practical performance and viability in larger networks.

In an effort to investigate sPAR's viability, this thesis aims to implement the scheme and conduct a structured evaluation of its performance with respect to the number of participants in the system.

Research Questions:

- What is the computational cost of sPAR, and how does it compare to the theoretical complexity described in the original paper?
- How does the performance of sPAR scale with the number of participants, and at what point does it become impractical?

Approved: 2026-04-01 – Associate Professor Park, Jeongeun, NTNU (Main supervisor)

Abstract

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

This is the second paragraph. Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

And after the second paragraph follows the third paragraph. Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Sammendrag

After this fourth paragraph, we start a new paragraph sequence. Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

This is the second paragraph. Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Preface

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Contents

List of Figures	ix
List of Tables	xi
List of Algorithms	xiii
1 Background	1
2 Related Works	3
2.1 TFHE Bootstrapping Primitives	3
2.1.1 Generalization	4
2.1.2 Noise	6
2.2 Residue Number Systems	6
2.2.1 Number Theoretic Transform	7
2.2.2 RNS Base Conversion	8
2.3 Gadgets in RNS (keyswitching)	10
2.3.1 Brakerski-Vaikuntanathan	11
2.3.2 Double Decomposition	11
2.3.3 Gentry-Halevi-Smart	12
2.3.4 Hybrid	13
2.4 RLWE-to-RGSW Expansion	14
2.4.1 ORAM	14
2.5 (Somewhat) Practical Anonymous Routing	16
2.5.1 Server Write	16
2.5.2 HomPlacing	16
3 Methodology	19
3.1 Code	19
3.2 Experimental Setup	19
3.3 Implementation Details	19
3.3.1 BV (Double Decomposition)	19
3.4 GHS/Hybrid	24

4	Results and Discussion	25
5	Conclusion and Future Work	27
	References	29
	Appendices	
A	Decomposition	31
B	Number Theory	33
	B.1 Chinese Remaineder Theorem	33
	B.2 Number Theoretic Transform	33

List of Figures

2.1	HomPlacing tree with a 3-bit index a . At level i , the i -th MSB of the index a determines homomorphically whether the ciphertext is placed in the left (0) or right (1) bucket. This process is repeated $\log n = 3$ times until the ciphertext is placed in the a -th bucket, without the placer ever knowing which bucket that is. The highlighted path shows the route taken for the index $a = 6 = [110]_2$: the MSB places the ciphertext in the right bucket, and so on. Again, the server doesn't know a , only the <i>encryption</i> of the bits $[a]_2$	17
-----	--	----

List of Tables

3.1	The $k \times \ell$ structure of the Double-CRT Decomposition $D_Q(c)$	22
-----	--	----

List of Algorithms

2.1	Base Extension	9
2.2	Fast Base Extension	9
2.3	Fast Base Extension $Q \rightarrow QP$ (ApproxModUp)	10
2.4	HomExpand	14
2.5	expandRlwe	15
2.6	expandRlwe	15
2.7	Homomorphic placing HomPlacing	16
2.8	Homomorphic placing HomPlacing for one slot	17
3.1	Encrypt RGSW-BV	21
3.2	Encrypt RGSW (optimized)	22
3.3	Optimized Branchless Signed LSB Decomposition ($B = 2^b$)	23

Chapter 1

Background

Anonymous communication aims to enable users to exchange messages over a network without revealing who is communicating with whom, thereby protecting both sender and receiver identities. Many such systems have been proposed and implemented over the years, such as Tor, I2P, and mixnets. Most of these designs depend on the *threshold model* to provide anonymity, requiring at least some components of their infrastructure to behave honestly. In practice, this assumption is increasingly strained by large-scale surveillance, correlation attacks, and the growing centralization of internet infrastructure [SEV+15].

Systems with stronger anonymity guarantees based on Multi-Party Communication (MPC) exist. However, these suffer from poor scalability due to their high interactivity, making them impractical for large-scale networks. A Dining Cryptographers network (DC-net), for example, can provide anonymity even if there are no trusted components in the network infrastructure. Doing so requires every participant to interact with every other participant, for every round of messages sent.

Recently, a new line of research has proposed fundamentally different designs, aiming to offer strong anonymity even in the presence of compromised or adversarial components in the network actively attempting to break anonymity. The first such design was the Non-Interactive Anonymous Router (NIAR) [SW21]. This paper proposed using Multi-Client Functional Encryption (MCFE) to allow a single, untrusted router to shuffle or permute messages from a set of clients *obliviously*; without learning anything about the permutation. Thereby making it infeasible for the router to link any of the messages to its sender.

NIAR was a breakthrough in anonymous communication, not only showing the theoretical possibility of designing anonymous routers, but also that such routers can be *non-interactive*: clients only send one message to the router and do not need to perform any additional interaction to achieve anonymity. Using a centralized (anonymous) router in this way yields similar benefits to existing MPC-based systems,

without needing any interactions between clients. However, there are a number of problems that makes NIAR impractical, if not impossible, to implement in practice.

1. The cryptographic components required by NIAR are extremely computationally expensive or not known to be implementable in practice.
2. Achieving anonymity for recipients assumes the existence of a very strong primitive called *indistinguishability obfuscation with sub-exponential security*. The practicality of such a construction is unknown.
3. Reusing the same setup across multiple rounds makes messages from the same sender linkable, unless the expensive setup phase is rerun for every round
 - Anonymous communication
 - Current solutions
 - Problems with current solutions
 - FHE as a new approach
 - sPAR
 - RSA is multiplicatively homomorphic, but (follow-up) paper theorized a *fully* homomorphic scheme may be possible [Gen09; RAD78]

Chapter 2

Related Works

This section details the fundamental building blocks of the sPAR protocol, and the protocol itself. To achieve a reasonable depth the protocol utilizes the *external product*, a primitive from TFHE bootstrapping that allows homomorphic multiplication of binary digits with near-constant noise bounds — enabling a large number of binary multiplication without consuming a level/bootstrapping. This primitive was ported to BGV-RNS for this thesis, which does not natively allow digit decomposition as normally used by the external product.

2.1 TFHE Bootstrapping Primitives

- TFHE bootstrapping achieves fast evaluation of binary circuits.
- It is based on (approximating) *gadget decomposition*.
- Asymmetry in noise scaling of GSW ciphertexts.
- Decomposition [Ko26]: *a mathematical technique used to represent a number in a smaller base (radix) while preserving its value* $\gamma = \gamma_0 \frac{q}{\beta_0} + \dots + \gamma_\ell \frac{q}{\beta_\ell}$
- Gadget [Ko26]: *generalized form of number decomposition* $\gamma = \gamma_0 g_0 + \dots + \gamma_\ell g_\ell$

Definition 2.1. RGSW Ciphertext A (Ring) GSW ciphertext (2.1).. where Z is a vector of 2ℓ (fresh) Ring Learning With Errors (RLWE) encryptions of 0. The message $m \in \{0, 1\}$, and $G = I_2 \otimes g$ where g is the *gadget vector*, typically defined as $g = (1, 1/B, \dots, 1/B^{\ell-1})$.

$$\text{RGSW}_s(m) = Z + m \cdot G \quad (2.1)$$

Definition 2.2. Gadget Property. Two mappings $D(m)$ and $P(n)$ have the gadget property (wrt. Q) iff.

$$\langle D(m), P(n) \rangle = m \cdot n \pmod{Q} \quad (2.2)$$

The external product approximates the gadget property by defining a gadget matrix G and inverse gadget matrix $G^{-1}(x)$ s.t.

$$G^{-1}(x) \cdot G = x + \epsilon \approx x \pmod{Q} \quad (2.3)$$

Definition 2.3. External Product $\boxtimes$ defines a product between an RLWE and an (Ring) Gentry-Sahai-Waters (RGSW) ciphertext. $G^{-1}(x)$ satisfies the approximate gadget property (2.3).

$$\text{RLWE}_s(a) \boxtimes \text{RGSW}_s(b) \rightarrow \text{RLWE}_s(a \cdot b) \quad (2.4)$$

$$a \boxtimes B = G^{-1}(a) \cdot B \pmod{Q} \quad (2.5)$$

Definition 2.4. Internal Product $\boxtimes$ defines a product between RGSW ciphertexts by repeated external products.

$$\text{RGSW}_s(a) \boxtimes \text{RGSW}_s(b) = \text{RGSW}_s(a \cdot b) \quad (2.6)$$

$$A \boxtimes B = \begin{bmatrix} A_0 \boxtimes B \\ \vdots \\ A_{2\ell-1} \boxtimes B \end{bmatrix} \quad (2.7)$$

2.1.1 Generalization

The TFHE scheme [CGGI18] explicitly uses the gadget vector $g = (1, 1/B, \dots, 1/B^\ell)$ **Dec algorithm**, but it should be noted that any choice of G and G^{-1} that satisfy the (inexact) gadget property (2.3) is sufficient to define a valid RGSW ciphertext. For later it will be particularly helpful to establish constructively that/how any pair of vectors that satisfy the gadget property (2.2) can be used to find valid gadget matrices.

Theorem 2.5. *Any pair $D(a), P(b)$ that satisfies the approximate gadget property (2.3) defines an RGSW ciphertext (2.1) and a valid external product (2.4) such that $\text{RLWE}(a) \boxtimes \text{RGSW}(b) = \text{RLWE}(a \cdot b)$. Further, a valid (inverse) gadget matrix is given by:*

$$G = I_2 \otimes P(1) \quad (2.8)$$

$$G^{-1}(x) = [D(x_0)|D(x_1)] \quad (2.9)$$

Proof. The inexact gadget property holds for $G = I_2 \otimes P(1)$ and $G^{-1}(x) = [D(x_0)|D(x_1)]$.

$$\begin{aligned} G^{-1}(x) \cdot G &= [D(x_0)|D(x_1)] \begin{pmatrix} P(1) & 0 \\ 0 & P(1) \end{pmatrix} \\ &= [D(x_0) \cdot P(1), D(x_1) \cdot P(1)] \\ &= [x_0 \cdot 1 + \epsilon_0, x_1 \cdot 1 + \epsilon_1] \\ &= [x_0, x_1] + [\epsilon_0, \epsilon_1] = x + \epsilon \pmod{Q} \end{aligned}$$

If the inexact gadget property holds, then the external product is valid.

$$G^{-1}(x) \cdot G = x + \epsilon_G \pmod{Q} \quad (2.10)$$

$$x \boxdot Y = G^{-1}(x) \cdot (Z + m_y \cdot G) \quad (2.11)$$

$$G^{-1}(x) \cdot Z + (G^{-1}(x) \cdot G)m_y \quad (2.12)$$

$$G^{-1}(x) \cdot Z + (x + \epsilon_G) \cdot m_y \quad (2.13)$$

$$G^{-1}(x) \cdot Z + x \cdot m_y + \epsilon_G \cdot m_y \quad (2.14)$$

$$(2.15)$$

The middle term $x \cdot m_y = (a \cdot s + m_x + \epsilon, a) \cdot m_y = ((am_y)s + (m_y m_x) + m_y \epsilon, am_y)$. By setting $a' = a \cdot m_y, \epsilon' = \epsilon \cdot m_y$ its clear that this becomes a valid RLWE encryption of $m_y \cdot m_x$ from $(a's + m_x \cdot m_y + \epsilon', a')$.

The left term $G^{-1}(x) \cdot Z$ evaluates to an RLWE encryption of 0 with some non-zero noise. (Here g_i^{-1} denotes the i -th element of $G^{-1}(x)$, not necessarily the inverse of $g \dots$)

$$G^{-1}(x) \cdot Z = \begin{pmatrix} g_0^{-1} & \cdots & g_{2\ell-1}^{-1} \end{pmatrix} \begin{pmatrix} b_0 & a_0 \\ \vdots & \vdots \\ b_{2\ell-1} & a_{2\ell-1} \end{pmatrix} \quad (2.16)$$

$$= \left(\sum_{i=0}^{2\ell-1} g_i^{-1} b_i, \sum_{i=0}^{2\ell-1} g_i^{-1} a_i \right) \quad (2.17)$$

Let $a_z = \sum_{i=0}^{2\ell-1} g_i^{-1} a_i$, and observe that the first element becomes

$$\sum_{i=0}^{2\ell-1} g_i^{-1} b_i = \sum_{i=0}^{2\ell-1} g_i^{-1} (a_i \cdot s + \epsilon_i) \quad (2.18)$$

$$\sum_i g_i^{-1} a_i \cdot s + \sum_i g_i^{-1} \epsilon_i \quad (2.19)$$

$$s \left(\sum_i g_i^{-1} a_i \right) + \sum_i g_i^{-1} \epsilon_i \quad (2.20)$$

$$a_z \cdot s + \epsilon_z \quad (2.21)$$

$$\therefore x \boxdot Y = \text{RLWE}_s(0) + \text{RLWE}_s(m_x \cdot m_y) + (\epsilon_G \cdot m_y) = \text{RLWE}_s(m_x \cdot m_y) \quad \square$$

Remark 2.6. The exact gadget property is a special case of the inexact gadget property with $\epsilon = 0$, so the same proof holds.

2.1.2 Noise

Each term is a source of noise:

- $\epsilon_z = \sum_i g_i^{-1} \epsilon_i$
- $\epsilon' = m_y \cdot \epsilon$
- $\epsilon_G \cdot m_y$

Some observations:

- Two error terms are multiplied by m_y , which is why this method works best in binary systems where $m_y \in \{0, 1\}$ as this error term is bounded by the original error term $\|\epsilon'\| \leq \|\epsilon\|$
- The decomposition makes ϵ_z small because g_i^{-1} are small and a sum of many small errors is better than a product of large errors (main point of decomposition). Also, since ϵ_i are sampled from a gaussian distribution, their expected value is 0.
- The ϵ_G comes purely from the choice of gadget. If we choose an *exact* gadget with $\epsilon = 0$ then $\epsilon_G \cdot m_y = 0$ and this error vanishes.
- g_i^{-1} are also from the gadget, but is less controllable as it also depends on x . A good choice of gadget is therefore one that satisfies the exact gadget property and has small elements in $G^{-1}(x)$.

The choice of gadget affects the noise and some gadgets are *better* than others, as they add less noise during the external product! Ideally we want to use a decomposition that has the *exact* gadget property, to minimize noise the noise from the “error (left) term”.

2.2 Residue Number Systems

- In BGV, BFV, CKKS, the ciphertext modulus q is large, much larger than a computer word (typically 64 bits on modern computers)
- Multi-precision arithmetic is known to be slow due to its serial nature
- A Residue Number System (RNS) represents numbers as a set of smaller residues of arbitrary size
- In practice: RNS versions of BGV, BFV and CKKS (e.g., OpenFHE only supports the RNS versions of these)

Definition 2.7. Canonical Form We are interested in polynomials in the ring $R_q = \mathbb{Z}_q/(X^N + 1)$ with degree (up to) $N - 1$ and coefficients in $[-q/2, q/2)$ (centered modulo representation), and different ways to represent this polynomial. The canonical form (2.22) of such a polynomial a is the form everyone is familiar with.

$$a = c_0 + c_1X + \cdots + c_NX^{N-1} = \sum_{i=0}^{N-1} c_iX^i \quad (2.22)$$

Definition 2.8. Coefficient Form As polynomials have a well-known/predictable structure, we can omit the redundant X variable and represent $a \in R_Q$ uniquely by its coefficients. I will refer to (only) this representation (2.23) as the **coefficient** form.

$$a = (c_0, c_1, \dots, c_N) \quad (2.23)$$

Let $Q = q_0 \dots q_{k-1}$ with $\gcd(q_i, q_j) = 1 \forall i \neq j$, so that the Chinese Remainder Theorem (CRT) allows decomposition into smaller residues. If $\max_i \{q_i\}$ fits in a computer word then we can use CRT on the coefficients of a polynomial to work exclusively with integers that fit inside a computer word.

Definition 2.9. RNS/CRT Form Given a polynomial $a \in R_q$ where $q = q_0 q_1 \dots q_{k-1}$ is chosen as a product of k small primes (small meaning they each fit inside a computer register), each coefficient c_i is uniquely represented by its k residues via the Chinese Remainder Theorem. Then the **CRT form** of a (2.24) is a collection of smaller polynomials $(\text{mod } p_i)$ called *towers*, *residues* or *limbs*.

$$\begin{pmatrix} a \bmod q_0 \\ a \bmod q_1 \\ \vdots \\ a \bmod q_{k-1} \end{pmatrix} = \begin{pmatrix} c_0 \bmod p_0 & c_1 \bmod p_0 & \dots & c_{N-1} \bmod p_0 \\ c_0 \bmod p_1 & c_1 \bmod p_1 & \dots & c_{N-1} \bmod p_1 \\ \vdots & \vdots & \ddots & \vdots \\ c_0 \bmod p_{k-1} & c_1 \bmod p_{k-1} & \dots & c_{N-1} \bmod p_{k-1} \end{pmatrix} \quad (2.24)$$

Informal: Due to the nature of modular arithmetic, addition/subtraction of two polynomials in R_Q is equivalent to adding/subtracting their CRT forms (modulo p_i). As each element fits within a computer word, each pair of elements can be added/subtracted in parallel.

Remark 2.10. Rather confusingly, OpenFHE calls this form *Coefficient* format. Both D/CRT forms are represented by an `DCRTPoly` object, so a polynomial in CRT form is represented by a `DCRTPoly` object in `Format::Coefficient` mode.

2.2.1 Number Theoretic Transform

- The Number Theoretic Transform (NTT) converts a complex convolution (specifically: cyclic or negacyclic convolution) in the coefficient (CRT?) domain into point-wise multiplication in the NTT domain.
- Polynomial multiplication is one such convolution.
- We apply NTT to the coefficient form (2.24)

Definition 2.11. Double CRT/NTT/Evaluation Form By applying the NTT to the CRT form (2.24) we gain the ability to perform polynomial multiplication in parallel, without losing the ability to perform addition/subtraction in parallel.

$$\begin{pmatrix} \text{NTT}(a \bmod q_0) \\ \text{NTT}(a \bmod q_1) \\ \vdots \\ \text{NTT}(a \bmod q_k) \end{pmatrix} \quad (2.25)$$

The evaluation form (2.25) is the preferred form, as it supports parallel addition, subtraction and multiplication, but it still does not natively support *all* necessary operations. Because switching between canonical/vector, CRT and NTT mode is relatively expensive — requiring (inverse) CRTs and NTTs — much of the relevant literature [GHS12; BEHZ16; HPS18; KPZ21] improves upon existing functions by finding new approaches and approximations that limit the number of format-switches required.

$$\text{Coefficient} \begin{matrix} \xrightarrow{\text{CRT}} \\ \xleftarrow{\text{iCRT}} \end{matrix} \text{RNS} \begin{matrix} \xrightarrow{\text{NTT}} \\ \xleftarrow{\text{iNTT}} \end{matrix} \text{DCRT}$$

2.2.2 RNS Base Conversion

In later sections we will need to efficiently express a polynomial in different *bases* i.e. moduli.

Definition 2.12. RNS Basis Given a large modulus $Q = q_0 q_1 \dots q_{k-1}$, the *RNS basis* $\mathcal{B}_Q = \{q_0, q_1, \dots, q_{k-1}\}$ denotes the moduli used in the RNS representation of $a \in R_Q$. A polynomial $a \in R_Q$ can then be expressed by its residues modulo the elements of $\mathcal{B}_Q$

$$a \in \mathcal{B}_Q \implies a \equiv r_0 \pmod{q_0}, \dots \quad (2.26)$$

Remark 2.13. To avoid confusion, I will denote a polynomial a in *coefficient mode* (2.23) as an element in the ring $a \in R_Q$ and a polynomial in *NTT mode* as an element of the base $a \in \mathcal{B}_Q$. If a polynomial is expressed in *CRT mode* (2.24), then it will be explicitly stated.

Base Extension

We have a polynomial $a \in \mathcal{B}_Q$ in evaluation form (2.25) and want to express the same polynomial in $\mathcal{B}_{QP}$, where $Q = q_0 \dots q_k$ and $P = p_0 \dots p_m$. P is chosen such that $\gcd(p_i, p_j) = 1$ for all $i \neq j$ and $\gcd(p_i, q_j) = 1$ for all p_i, q_j (as well as

$\gcd(q_i, q_j) = 1 \forall i \neq j$). As Q and P are disjoint, QP shares the q residues so we do not need to re-calculate them.

Algorithm 2.1 Base Extension

Input: $a \in \mathcal{B}_Q, \mathcal{B}_P$

Output: $a \in \mathcal{B}_{QP}$

- 1: Initialize an empty array a_P of size $\|\mathcal{B}_P\|$ $\triangleright$ Holds the P -towers
 - 2: $a_{\text{coeffs}} := \text{InverseCRT}(\text{InverseNTT}(a))$ $\triangleright$ Convert to coefficient form
 - 3: **for** $p_i \in \mathcal{B}_P$ **do**
 - 4: $a_{p_i} := a_{\text{coeffs}} \pmod{p_i}$ $\triangleright$ Find p_i -tower
 - 5: Append $\text{NTT}(a_{p_i})$ to a_P
 - 6: **end for**
 - 7: **return** $a \cup a_P$
-

Fast Base Extension

The base extension requires converting an NTT polynomial $a \in \mathcal{B}_Q$ to CRT mode, then coefficient mode. The improvements made by [BEHZ16] performs the base extension in the CRT representation, saving one inverse CRT at the cost of some additional noise.

$$\text{FastBConv}(x, Q, B) = \left(\sum_{i=1}^k \left| x_i \frac{q_i}{Q} \right|_{q_i} \times \frac{Q}{q_i} \right)_{m \in B} \quad (2.27)$$

Algorithm 2.2 Fast Base Extension

Input: $a \in \mathcal{B}_Q, \mathcal{B}_P$

Output: $(a + \epsilon_P) \in \mathcal{B}_{QP}$, where $\epsilon_P = \alpha_x Q$ for some $\alpha_x \in [0, k)$

- 1: $a_Q := \text{InverseNTT}(a)$ $\triangleright$ Convert to CRT form
 - 2: $a_P := \text{FastBConv}(a_Q, \mathcal{B}_Q, \mathcal{B}_P)$
 - 3: **return** $a \cup \text{NTT}(a_P)$
-

Remark 2.14. Note that the factors $| \frac{q_i}{Q} |_{q_i}$ and $\frac{Q}{q_i}$ can be pre-calculated and stored in memory (will be discussed in ??).

The fast base extension calculates $x + \alpha_x q$ for some integer $\alpha_x \in [0, k - 1]$. The paper also shows how to correct this error...

Modulus Switching

Also referred to as *mod down*

Algorithm 2.3 Fast Base Extension $Q \rightarrow QP$ (ApproxModUp)

Input: $x = \{x_i\}_{i=0}^k$, with $x_i = x \bmod q_i$, in base $Q = \{q_0, \dots, q_k\}$ **Input:** Precomputed $\tilde{q}_i = \left(\frac{Q}{q_i}\right)^{-1} \bmod q_i$ for $i = 0, \dots, k$ **Input:** Precomputed $\omega_{i,\ell} = \frac{Q}{q_i} \bmod p_\ell$ (Shoup form) for $i = 0, \dots, k, \ell = 0, \dots, j$ **Output:** $x' = \{x'_s\}_{s \in QP}$, the residues of $x + uQ$ in base $QP = \{q_0, \dots, q_k, p_0, \dots, p_j\}$,
 $\|u\|_\infty < \frac{k+1}{2}$

1: // shared source-side residues (hoisted, target-independent)

2: for $i = 0$ to k do3: $y_i \leftarrow x_i \cdot \tilde{q}_i \bmod q_i$

4: end for

5: // Q -part: reuse input limbs, zero cost, overflow-free6: for $i = 0$ to k do7: $x'_{q_i} \leftarrow x_i$

8: end for

9: // P -part: the only arithmetic, with lazy reduction10: for $\ell = 0$ to j do11: $\text{acc} \leftarrow 0$

▷ wide / unreduced accumulator

12: for $i = 0$ to k do13: $\text{acc} \leftarrow \text{acc} + y_i \cdot \omega_{i,\ell}$

▷ ShoupMulAdd, deferred reduction

14: end for

15: $x'_{p_\ell} \leftarrow \text{acc} \bmod p_\ell$

16: end for

17: return x'

Approximate Mod Down $QP \rightarrow Q$

??? [GHS12]

2.3 Gadgets in RNS (keyswitching)

Digit decomposition requires a to be in its coefficient representation (2.23), does not natively work on residues and is therefore not well suited for RNS. A few RNS-native gadgets have been proposed over the years, primarily in the context of key switching, such as [BEHZ16; HPS18] based on the BV gadget, and [GHS12] based on RNS-base extension.

2.3.1 Brakerski-Vaikuntanathan

Although we cannot use *digit* decomposition, RNS is itself based on a different decomposition: *CRT* decomposition. This means we can decompose an RNS polynomial using the RNS primes themselves as a basis (where the $\mathcal{B}_Q$ notation usually comes from! as shown in [BEHZ16; KPZ21]).

$$D_Q(a) = \left(\left[a \left(\frac{Q}{q_0} \right)^{-1} \right]_{q_0}, \dots, \left[a \left(\frac{Q}{q_i} \right)^{-1} \right]_{q_i} \right) \quad (2.28)$$

$$P_Q(a) = \left(\left[a \frac{Q}{q_0} \right]_Q, \dots, \left[a \frac{Q}{q_i} \right]_Q \right) \quad (2.29)$$

Lemma 2.15. *The vectors (2.28) and (2.29) satisfy the gadget property [KPZ21].*

$$\langle D_Q(a), P_Q(b) \rangle = a \cdot b \pmod{Q} \quad (2.30)$$

It was later shown by Halevi, Polyakov and Shoup [HPS18] that the factors can be moved into $P_Q(a)$ without violating Theorem 2.15. Since $[(Q/q_i)^{-1}]_{q_i} [Q/q_i] = 1$ in tower $j = i$ and 0 in towers $j \neq i$, this optimization effectively makes the decomposition free as $P_Q(a) \cong a_{\text{limbs}} \cdot I_k$ where k is the number of RNS limbs.

$$\begin{aligned} D_Q(a) &= ([a]_{q_0}, [a]_{q_1}, \dots, [a]_{q_i}) \\ P_Q(a) &= \left(a \cdot \left[\left(\frac{Q}{q_0} \right)^{-1} \right]_{q_0} \cdot \left[\frac{Q}{q_0} \right]_Q, \dots, a \cdot \left[\left(\frac{Q}{q_i} \right)^{-1} \right]_{q_i} \cdot \left[\frac{Q}{q_i} \right]_Q \right) \end{aligned}$$

2.3.2 Double Decomposition

By Theorem 2.5, we can use these gadget vectors to define an external product. (Also optimized heavily in ??.) However, the noise added scales with $\sum_i q_i/2 \leq k \cdot \max_i \{q_i\}/2$ (Cite error bounds from TFHE paper). This is fine when only applied once, for example in keyswitching, but quickly becomes unmanageable when using *internal* products (2.6).

To combat this, we can add a second layer of decomposition using the classic BV technique with $g = (1, B, \dots, B^{\ell-1})$. I started this section by saying this method is slow, and that still rings true as digit decomposition cannot be performed in RNS/DCRT mode. However, due to the gadgets, the coefficients of a **are** the coefficient/canonical representations of the smaller polynomials $a \pmod{q_i}$. By applying the technique to each RNS digit individually, we:

1. avoid multi-precision arithmetic even in $D_Q(a)$
2. convert to RNS coefficient mode instead of canonical (standard coefficient)
3. can apply $P_Q(a)$ fully in evaluation mode

Corollary 2.16. **Correctness of Double Decomposition** *Applying digit decomposition to each element of $P_{\text{HPS}}(a)$ individually yields a valid external product by Theorem 2.5.*

Does not matter whether the decomposition is first or last $\text{Decomp}(P_{\text{HPS}}(a))$ or $P_{\text{HPS}}(\text{Decomp}(a))$

Proof. Let $P_{\text{BV}}(a) = P_{\text{HPS}}(a \cdot g)$ and $D_{\text{BV}}(a) = D_{\text{HPS}}(a \cdot g^{-1})$ where $g = ()$ and g^{-1} is the digit extraction of n such that $g^{-1}(n) \cdot g = n$. Using the fact that $P_{\text{HPS}}(\cdot), D_{\text{HPS}}(\cdot)$ satisfies the exact gadget property (2.2), we see that the same is true for $P_{\text{BV}}(\cdot), D_{\text{BV}}(\cdot)$:

$$\langle P_{\text{BV}}(m), D_{\text{BV}}(n) \rangle = \langle P_{\text{HPS}}(m \cdot g), D_{\text{HPS}}(n \cdot g^{-1}) \rangle \pmod{Q} \quad (2.31)$$

$$= (m \cdot g) \cdot (n \cdot g^{-1}) = m \cdot n \cdot (g^{-1} \cdot g) \pmod{Q} \quad (2.32)$$

$$= m \cdot n \pmod{Q} \quad (2.33)$$

$$\therefore \langle P_{\text{BV}}(m), D_{\text{BV}}(n) \rangle = m \cdot n \pmod{Q} \quad \square$$

Remark 2.17. *Now ℓ covers $\max_i \{q_i\}$ and the RGSW has $k \cdot \ell$ rows. This is the same length required if ℓ covers the entire Q as we would have k less numbers but each number would be k digits longer. For example, with $\ell_{\text{small}} = 2$ and $Q = q_0 q_1 \implies k = 2$ we get $2 \cdot k \cdot \ell_{\text{small}} = 2 \cdot 2 \cdot 2 = 8$ RGSW rows and the noise is equivalent to having $\ell_{\text{big}} = 4 \implies 2 \cdot \ell_{\text{big}} = 8$ RGSW rows over Q . The results is the same, the number of rows is the same, the noise is the same, its just faster to calculate over each q_i individually.*

2.3.3 Gentry-Halevi-Smart

The GHS technique, first introduced by Gentry, Halevi and Smart in [GHS12], is a faster method of performing keyswitching in RNS than the previous method. Later the method was improved by the BEHZ authors [BEHZ16]. At a high level, the GHS technique involves three steps:

1. Generate a public key $\text{Enc}(b \cdot P)$ in QP
2. Lift the ciphertext $\text{Enc}(a)$ into a higher basis $Q \rightarrow QP$

3. Apply the gadget operation in QP to get $\text{Enc}(a \cdot b \cdot P) + \epsilon \pmod{QP}$
4. Modulus switch the result $QP \rightarrow Q$ to get $\text{Enc}(a \cdot b) + \epsilon/P \pmod{Q}$

The relevant result of [GHS12] for our purpose was finding a way to approximate modulus switching in the evaluation representation of a polynomial, and using this to devise the GHS keyswitching technique. Later, [BEHZ16] proposed a fast approximation to the base extension/lifting, operating on the CRT representation of a polynomial. The *extra* noise from the operation is divided by P in the “mod down”, so the noise added by the technique itself $\epsilon \approx |\frac{Q}{P}|$ is approximately 1 if Q and P are chosen to be the same bit-length ($\implies 1 \times \epsilon_{ks}$).

Gadget decomposition

Does not exactly fit inside the established framework as $\langle D(x), D'(y) \rangle = x \cdot y \cdot P \pmod{Q}$ and the P is removed as a separate step in the external product.

2.3.4 Hybrid

Combines the best of both worlds: (single-layer) BV/BEHZ/HPS and GHS. Tunable with d_{num} where $|P| \approx |Q|/d_{num}$. I use $d_{num} = 1$ in practice, turning the hybrid scheme into (pure) GHS.

2.4 RLWE-to-RGSW Expansion

2.4.1 ORAM

As established in ??, RGSW ciphertexts are enticing because the external product allows us to perform (binary) homomorphic multiplication without relinearization. However, the size of RGSW makes the communication costs very high in a communication protocol. We wish to strike a balance between the cheap communication cost of packed RLWE and the fast multiplication of RGSW. The solution is to send RLWE ciphertexts from the client, and *expand* them homomorphically on the server.

Solution from [CCR19]. **HomExpand**: Expands an RLWE ciphertext into an RGSW ciphertext.

This expansion is described in Algorithm 4 of [CCR19], which builds on Algorithms 3 and 8 from the same paper. The algorithms are listed below (in slightly different notation) as Algorithm 2.4, Algorithm 2.5, and ??, respectively.

Algorithm 2.4 HomExpand

```

1: Input:  $\mathbf{c}_k = \text{RLWE}(\sum_{i=0}^{n-1} \frac{b_i}{nB^{k+1}})$ ,  $b_i \in \{0, 1\}$ ,  $0 \leq k < \ell$ 
2: Input:  $A = \text{RGSW}(-s)$ 
3: Output:  $\mathbf{C}_i = \text{RGSW}(b_i)$ ,  $0 \leq i < n$ 
4: for each  $k$  do
5:    $\mathbf{c}'_k := \text{expandRlwe}(c_k)$ 
6: end for
7: for  $i = 0, i < n$  do
8:   Initialize  $\mathbf{C}_i$  as an empty  $2\ell \times 2$  matrix of polynomials
9:   for  $k = 0; k < \ell$  do
10:     $\mathbf{C}_i[k] = A \boxtimes \mathbf{c}'_k[i]$ 
11:     $\mathbf{C}_i[k + \ell] = \mathbf{c}'_k[i]$ 
12:   end for
13: end for
14: return  $\{\mathbf{C}_j\}_{j=0}^{n-1}$ 

```

Algorithm 2.4 takes as input ℓ RLWE ciphertexts, which in sPAR correspond to the ℓ encrypted bits of the index a_j . It also takes an RGSW ciphertext of the encrypted secret key

It begins by running a subroutine `expandRlwe` (Algorithm 2.5) which converts each of the ℓ RLWE ciphertexts into a matrix, like a *partial* RGSW. Then these partial RGSW's are combined into a single (complete) RGSW.

This subroutine converts an RLWE ciphertext with ℓ slots into ℓ ciphertexts, each encoding a unique slot of the input. **Subs** is a function that... (EvalAutomorphism!)

Algorithm 2.5 expandRlwe

```

1: Input:  $\mathbf{c} = \text{RLWE}(\sum_{i=0}^{n-1} b_i X^i)$ ,  $b_i \in \{0, 1\}$ 
2: Output:  $\mathbf{c}_i = \text{RLWE}(n \cdot b_i)$ ,  $0 \leq i < n$ 
3:  $\mathbf{c}_0 := \mathbf{c}$ 
4: for  $i = 0$ ,  $i \leq \log n$  do
5:    $k = n/2^{i-1} + 1$ 
6:   for  $b = 0; b < 2^i$  do
7:      $\mathbf{c}_{2b} = c_b + \text{Subs}(c_b, k)$ 
8:      $\mathbf{c}_{2b+1} = c_b - \text{Subs}(c_b, k)$ 
9:   end for
10: end for
11: return  $\{\mathbf{c}_j\}_{j=0}^{n-1}$ 

```

Their method is fast for TFHE, but for our use-case, where $n \ll N$, it's faster to use the method described in [HS18] to repeatedly “rotate” the ciphertext ℓ times as in Algorithm 2.6. The FastRotate operation is supported by the OpenFHE as EvalFastRotate.

Note that the components being scaled by n is a side-effect of their implementation, and not required when using FastRotate, so the output is a list of $c_i = \text{RLWE}(b_i)$.

Algorithm 2.6 expandRlwe

```

1: Input:  $\mathbf{c} = \text{RLWE}(\sum_{i=0}^{n-1} b_i X^i)$ ,  $b_i \in \{0, 1\}$ 
2: Output:  $\mathbf{c}_i = \text{RLWE}(b_i)$ ,  $0 \leq i < n$ 
3:  $\mathbf{c}_0 := \mathbf{c}$ 
4: for  $i = 0$ ,  $i \leq \log n$  do
5:   do something...
6: end for
7: return  $\{\mathbf{c}_j\}_{j=0}^{n-1}$ 

```

2.5 (Somewhat) Practical Anonymous Routing

The main protocol.

2.5.1 Server Write

Algorithm 2.7 Homomorphic placing HomPlacing

```

1: let notHasWritten = RGSW*(1)
2: for  $d \leftarrow \{0, 1, 2\}$  do
3:   for  $k \leftarrow \{0, 1, 2\}$  do
4:     for  $i \leftarrow \{0, \dots, \eta - 1\}$  do
5:       let  $h = z_{i,d} \boxtimes I_{i,k} \boxtimes \text{notHasWritten}$ 
6:        $L_{i,k} := L_{i,k} + h \cdot V_r$ 
7:        $I_{i,k} := I_{i,k} - h$ 
8:       notHasWritten := notHasWritten -  $h$ 
9:     end for
10:   end for
11: end for
12: return notHasWritten

```

* The server has no obligation to encrypt its values ($\text{notHasWritten}, L, I$) and may in some cases even be actively trying to break privacy and thus cannot be trusted to add any noise. This improves the performance of the algorithm as noiseless “encryption” is marginally faster as it does not need to perform any random sampling, and makes the algorithm as a whole more tolerable to noise.

2.5.2 HomPlacing

Algorithm 1 shows the idea of Algorithm 2.

- From NIAR: sorting of random numbers = shuffling.
- Evaluate tree, each bit determines if the data is placed left or right. One level per bit means there are 2^n buckets in the end, each one representing a bit string i.e. 000, 001, 010, 011, 100, 101, 110, 111 for 3 bits.
- We can encode the bits into RGSW form.

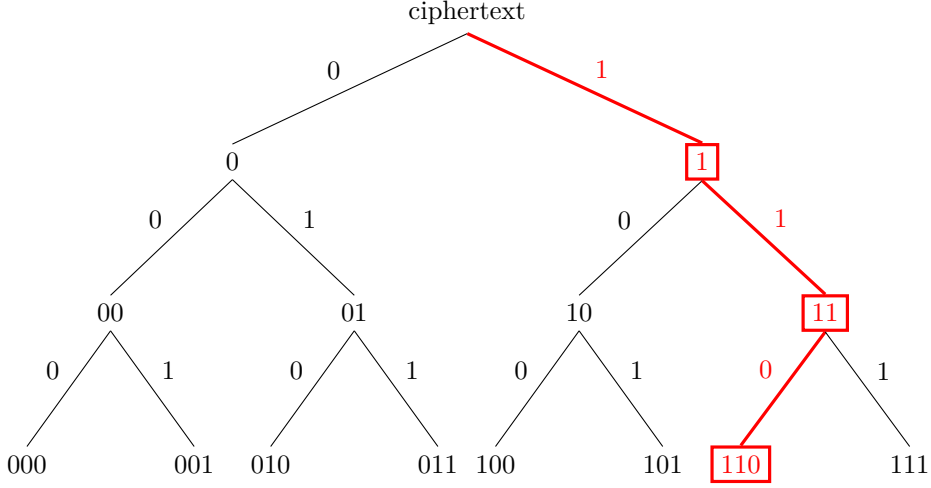


Figure 2.1: HomPlacing tree with a 3-bit index a . At level i , the i -th MSB of the index a determines homomorphically whether the ciphertext is placed in the left (0) or right (1) bucket. This process is repeated $\log n = 3$ times until the ciphertext is placed in the a -th bucket, without the placer ever knowing which bucket that is. The highlighted path shows the route taken for the index $a = 6 = [110]_2$: the MSB places the ciphertext in the right bucket, and so on. Again, the server doesn't know a , only the *encryption* of the bits $[a]_2$.

Algorithm 2.8 Homomorphic placing HomPlacing for one slot

- 1: **Input:** An encrypted value V_r and $\log n$ bits $\{c_0, \dots, c_{L-1}\}$, where $L = \log n$
 - 2: **Output:** A vector $\mathbf{l}$ consisting of n values of leaves: $z_0, \dots, z_{n-1}$, where $\mathbf{l}[i] = z_i$ for $i \in \{0, \dots, n-1\}$
 - 3: $\mathbf{l} := \mathbf{0}$
 - 4: Initialize root: $b_0 = V_r$
 - 5: Initialize the node values: $b_i := 0$ for $i \in \{1, \dots, 2n-2\}$
 - 6: **for** $i \leftarrow 0, \dots, L-1$ **do**
 - 7: **for** $j \leftarrow 0, \dots, 2^i-1$ **do**
 - 8: $r := b[2^i-1+j]$
 - 9: $b_{2^{i+1}+2j} := r \boxplus c_i$
 - 10: $b_{2^{i+1}+2j-1} := r - r \boxplus c_i$
 - 11: **end for**
 - 12: **end for**
 - 13: **for** $i \leftarrow 0, \dots, n-1$ **do**
 - 14: $z_i := b_{2^{L+1}-1+i}$
 - 15: $\mathbf{l}[i] = z_i$
 - 16: **end for**
 - 17: **return** $\mathbf{l}$
-

Chapter 3

Methodology

3.1 Code

Split into three sections found in the `lib` directory:

1. `core` extends OpenFHE's `CryptoContext<T>` with the RGSW-like operations:
 - `EncryptRGSW` outputs the RGSW encryption of the input plaintext
 - `EvalExternalProduct` takes an RLWE and RGSW ciphertext
 - `EvalInternalProduct` takes two RGSW ciphertexts
2. `server` builds the server functions on top of `core`
3. `client` builds the client functions on top of `core`

There are also two auxiliary directories `benchmarks` and `tests` that contain code to benchmark and unit test the functions, respectively.

3.2 Experimental Setup

...

3.3 Implementation Details

The library has two concrete implementations of the `ExtendedCryptoContext` base class, one based on the BV approach and one based on the GHS approach.

3.3.1 BV (Double Decomposition)

This is the closest to TFHE. As HPS is too noisy, we use digit decomposition on each *tower* in parallel (rather than the true polynomial). Hence we call this “*double decomposition*”.

Power of Base

As remarked earlier ??, the HPS optimization makes the CRT decomposition free as it is already required by the RNS/NTT system we are working in. Thus, layering in another decomposition (in this case digit decomposition) only requires performing the second decomposition on each tower/limb of the RNS representation.

$$\begin{aligned}
 P_{\text{HPS}}(a) &\cong \begin{bmatrix} [a]_{q_0} & 0 & \cdots & 0 \\ 0 & [a]_{q_1} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & [a]_{q_k} \end{bmatrix} = a \cdot I_k \\
 \text{compact storage} \quad \swarrow & \\
 \begin{bmatrix} [a]_{q_0} \\ [a]_{q_1} \\ \vdots \\ [a]_{q_k} \end{bmatrix} &= \begin{bmatrix} c_0 \bmod p_0 & c_1 \bmod p_0 & \cdots & c_{N-1} \bmod p_0 \\ c_0 \bmod p_1 & c_1 \bmod p_1 & \cdots & c_{N-1} \bmod p_1 \\ \vdots & \vdots & \ddots & \vdots \\ c_0 \bmod p_k & c_1 \bmod p_k & \cdots & c_{N-1} \bmod p_k \end{bmatrix} \\
 \text{decompose} \quad \downarrow & \\
 P_{\text{BV}}(a) &\cong \begin{bmatrix} [\hat{a}]_{q_0} \\ [\hat{a}]_{q_1} \\ \vdots \\ [\hat{a}]_{q_k} \end{bmatrix} = \begin{bmatrix} [a]_{q_0} & [aB]_{q_0} & \cdots & [aB^{\ell-1}]_{q_0} \\ [a]_{q_1} & [aB]_{q_1} & \cdots & [aB^{\ell-1}]_{q_1} \\ \vdots & \vdots & \ddots & \vdots \\ [a]_{q_k} & [aB]_{q_k} & \cdots & [aB^{\ell-1}]_{q_k} \end{bmatrix} \\
 \text{equivalent} \quad \updownarrow & \\
 \begin{bmatrix} [a \cdot g]_{q_0} \\ [a \cdot g]_{q_1} \\ \vdots \\ [a \cdot g]_{q_k} \end{bmatrix} &\cong \begin{bmatrix} [a]_{q_0} & [aB]_{q_0} & \cdots & [aB^{\ell-1}]_{q_0} \\ [a]_{q_1} & [aB]_{q_1} & \cdots & [aB^{\ell-1}]_{q_1} \\ \vdots & \vdots & \ddots & \vdots \\ [a]_{q_k} & [aB]_{q_k} & \cdots & [aB^{\ell-1}]_{q_k} \end{bmatrix} \\
 &\swarrow \quad \text{NTT} \\
 &\left(a, aB, \dots, aB^{\ell-1} \right)_{\text{DCRT}}
 \end{aligned}$$

Note that $\hat{a}$ is a vector of polynomials (the gadget decomposition of a), a is a polynomial, and c_i are the coefficients of a . When we apply the second decomposition, $a \rightarrow \hat{a}$, we get a 3-dimensional matrix of residues $\iff$ a 2-dimensional matrix of polynomial residues $\iff$ a vector of RNS polynomials $\therefore P_{\text{BV}}(a) \leftrightarrow (a, aB, \dots, aB^{\ell-1})$.

Then constructing $P_Q(a)$ is in some way equivalent to constructing $\hat{a}$, and the only difference between $P_Q(a)$ and $\hat{a}$ in practice are the indices of the elements as we ignore the 0-towers. In other words we can construct $\hat{a}$ and in the external product skip any computations that involves a 0-tower.

Definition 3.1. RGSW-BV Data Structure We define a data structure representing an RGSW ciphertext as a vector of RLWE ciphertexts. Let $G = I_2 \otimes P_{\text{BV}}(1) = I_2 \otimes P_{\text{HPS}}(g)$ and $G^{-1}(x) = [D_{\text{HPS}}(x_0)|D_{\text{HPS}}(x_1)]$ using the BV gadgets...

$$C = Z + mG = Z + m \begin{pmatrix} P(1) & 0 \\ \vdots & \vdots \\ P(1) & 0 \\ 0 & P(1) \\ \vdots & \vdots \\ 0 & P(1) \end{pmatrix} \quad (3.1)$$

Let $m_0 = m_0 \cdot P(1)$

Algorithm 3.1 Encrypt RGSW-BV

Input: Ring element m in NTT form and decomposition parameter ℓ

Output: RGSW as matrix of $2 \times k\ell$ polynomials

Definition 3.2. RGSW-BV External Product To calculate the external product using the BV decomposition using the RNS basis $\mathcal{B}_Q$ and decomposition parameter ℓ , we

And the external product:

$$x \boxdot Y = G^{-1}(x) \cdot Y \quad (3.2)$$

$$G^{-1}(x)(Z + m_y \cdot G) \quad (3.3)$$

$$G^{-1}(x) \cdot Z + m_y \cdot (G \cdot G^{-1}(x)) \quad (3.4)$$

$$G^{-1}(x) \cdot Z + m_y \cdot x \quad (3.5)$$

For the most optimal result, we flatten the loop to allow effective parallelization of each tower:

Algorithm 3.2 Encrypt RGSW (optimized)**Input:** Polynomial m and param ℓ **Output:** RGSW as matrix of $2 \times k\ell$ polynomials

```

1: Initialize a vector  $\mathbf{C}$  of  $2 \cdot k \cdot \ell$  RLWE ciphertexts
2: for  $row \in [0, 2 \cdot k \cdot \ell)$  in parallel do
3:    $h \leftarrow \lfloor row / (k\ell) \rfloor$ 
4:    $j \leftarrow \lfloor (row \bmod k\ell) / \ell \rfloor$ 
5:    $i \leftarrow \lfloor (row \bmod k\ell) \bmod \ell \rfloor$ 
6:    $z \leftarrow \text{RLWE}(0) = (c_0, c_1)$ 
7:    $c_h[j] \leftarrow [c_h[j] + mB^i]_{q_j}$ 
8:    $\mathbf{C}[h k \ell + j \ell + i] = z$ 
9: end for
10: return  $\mathbf{C}$ 

```

Decomposition

Digit extraction is performed in the coefficient domain. This is the main inefficiency in the HPS/BV-RNS gadget scheme. Let $\check{a} = ([d_0]_B, [d_1]_B, \dots, [d_{k-1}]_B)$ denote the decomposition of a using $??$. As $D_Q(a) = ([a]_{q_0}, [a]_{q_1}, \dots, [a]_{q_i}) = a$, the decomposition into $D_Q(\check{a})$ is equivalent to finding $(D_Q(d_0), D_Q(d_1), \dots, D_Q(d_{k-1}))$.

	Digit 0 (B^0)	Digit 1 (B^1)	$\dots$	Digit $\ell - 1$ ($B^{\ell-1}$)
Tower q_0	$d_{0,0}$	$d_{0,1}$	$\dots$	$d_{0,\ell-1}$
Tower q_1	$d_{1,0}$	$d_{1,1}$	$\dots$	$d_{1,\ell-1}$
$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$
Tower q_{k-1}	$d_{k-1,0}$	$d_{k-1,1}$	$\dots$	$d_{k-1,\ell-1}$

Table 3.1: The $k \times \ell$ structure of the Double-CRT Decomposition $D_Q(c)$.

Modulo Powers-of-2

The modulo operation is slow because it cannot be done in parallel. If we assume $B = 2^b$ is a power of 2 — which we can enforce by accepting $\log(B) = b$ as the input parameter instead of B — then we can use the fact that right-shift $(a \gg b) = a \bmod 2^b$ or something to make the modulo operation into a bit-wise (and thus easily parallelizable) operation.

Algorithm 3.3 Optimized Branchless Signed LSB Decomposition ($B = 2^b$)

Input: Coefficient $a \in \mathbb{Z}$, bit-width b , number of digits ℓ

Output: Vector of signed digits $\mathbf{d} = (d_0, d_1, \dots, d_{\ell-1})$ where $d_j \in [-2^{b-1}, 2^{b-1})$

```

1:  $a' \leftarrow a$ 
2:  $\text{mask} \leftarrow 2^b - 1$ 
3: for  $j = 0$  to  $\ell - 1$  do
4:    $u \leftarrow a' \& \text{mask}$                                  $\triangleright$  Extract the lowest  $b$  bits
5:    $\text{carry} \leftarrow u \gg (b - 1)$                          $\triangleright$  Extract the highest bit of  $u$  (1 or 0)
6:    $d_j \leftarrow u - (\text{carry} \ll b)$                        $\triangleright$  Branchlessly shift to negative range
7:    $a' \leftarrow (a' \gg b) + \text{carry}$                        $\triangleright$  Shift down and add the carry
8: end for
9: return  $\mathbf{d}$ 

```

3.4 GHS/Hybrid

- Reuses OpenFHE internals, including Hybrid keyswitching settings
- Downside: parameters tied to keyswitching, can be a problem if a program uses both keyswitching and RGSW

Pre-computed Values

For BV:

- $B^i \bmod q_j$ for all i, j
- Get $B = \lceil \max_j \|q_j\| / \hat{\ell} \rceil$ from ℓ

For GHS/Hybrid:

–

Chapter 4

Results and Discussion

- Parameters used and time per user/message of each step in the protocol
- Differences between BV and GHS gadgets for the external product
- AVX512 improvements

Chapter 5

Conclusion and Future Work

Conclusion

- Achieved better time than expected

Future Work

- BGV SIMD capabilities [] not utilized

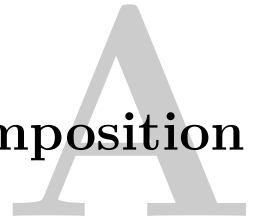
References

- [BEHZ16] J.-C. Bajard, J. Eynard, *et al.*, *A full RNS variant of FV like somewhat homomorphic encryption schemes*, Cryptology ePrint Archive, Paper 2016/510, 2016. [Online]. Available: <https://eprint.iacr.org/2016/510>.
- [CCR19] H. Chen, I. Chillotti, and L. Ren, “Onion ring oram: Efficient constant bandwidth oblivious ram from (leveled) tfhe”, in *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS ’19, London, United Kingdom: Association for Computing Machinery, 2019, pp. 345–360. [Online]. Available: <https://doi.org/10.1145/3319535.3354226>.
- [CGGI18] I. Chillotti, N. Gama, *et al.*, *TFHE: Fast fully homomorphic encryption over the torus*, Cryptology ePrint Archive, Paper 2018/421, 2018. [Online]. Available: <https://eprint.iacr.org/2018/421>.
- [Gen09] C. Gentry, “A fully homomorphic encryption scheme”, Ph.D. dissertation, Stanford University, 2009. [Online]. Available: <https://crypto.stanford.edu/craig>.
- [GHS12] C. Gentry, S. Halevi, and N. P. Smart, *Homomorphic evaluation of the AES circuit*, Cryptology ePrint Archive, Paper 2012/099, 2012. [Online]. Available: <https://eprint.iacr.org/2012/099>.
- [HPS18] S. Halevi, Y. Polyakov, and V. Shoup, *An improved RNS variant of the BFV homomorphic encryption scheme*, Cryptology ePrint Archive, Paper 2018/117, 2018. [Online]. Available: <https://eprint.iacr.org/2018/117>.
- [HS18] S. Halevi and V. Shoup, *Faster homomorphic linear transformations in HELib*, Cryptology ePrint Archive, Paper 2018/244, 2018. [Online]. Available: <https://eprint.iacr.org/2018/244>.
- [Ko26] R. Ko, *The beginner’s textbook for fully homomorphic encryption*, 2026. [Online]. Available: <https://arxiv.org/abs/2503.05136>.
- [KPZ21] A. Kim, Y. Polyakov, and V. Zucca, *Revisiting homomorphic encryption schemes for finite fields*, Cryptology ePrint Archive, Paper 2021/204, 2021. [Online]. Available: <https://eprint.iacr.org/2021/204>.
- [RAD78] R. L. Rivest, L. Adleman, and M. L. Dertouzos, “On data banks and privacy homomorphisms”, in *Foundations of Secure Computation*, 1978, pp. 169–180.

- [SEV+15] Y. Sun, A. Edmundson, *et al.*, “RAPTOR: Routing attacks on privacy in tor”, in *24th USENIX Security Symposium (USENIX Security 15)*, Washington, D.C.: USENIX Association, Aug. 2015, pp. 271–286. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity15/technical-sessions/presentation/sun>.
- [SW21] E. Shi and K. Wu, *Non-interactive anonymous router*, Cryptology ePrint Archive, Paper 2021/435, 2021. [Online]. Available: <https://eprint.iacr.org/2021/435>.

Appendix

Decomposition



Generally considered “*well-known*” in the literature, but stated here to avoid confusion. Following the definitions from [Ko26], a “*decomposition is a mathematical technique used to represent a number in a smaller base (radix) while preserving its value.*”

Definition A.1. Digit Decomposition A

Definition A.2. Gadget Decomposition B

Definition A.3. Gadget Property C

CRT is a decomposition!

Appendix

B

Number Theory

B.1 Chinese Remaineder Theorem

Could be stated here just to be more complete

B.2 Number Theoretic Transform

- Turns a convolution into a nega/cyclic
- Same as FFT but over different ring